

Semana 5

Selenium

Iniciamos esta semana viendo lo que son las bases de la herramienta de pruebas automatizadas Selenium para webpages, específicamente la versión en base de java. Vimos los conceptos base y principios, y elaboramos un proyecto demostrando estos mismos conceptos aplicados.

En mi evidencia, vemos como tenemos un proyecto base en donde importamos las herramientas que nos dejen trabajar con Selenium, que en mi caso fue Selenium para Eclipse IDE. También, para tener un manejo más fácil de código de pruebas y elaboración de reportes se usó TestNG, framework open source de pruebas para Java.

La estructura del código está sencilla, principalmente dividida en dos partes, la parte base que nos maneja la lógica interna de las webpages que vamos a probar, y la parte test que ya especifica exactamente que es lo que queremos comprobar. Por último, también hay una sección de utilidades, que nos da la lógica para la creación de la instancia del web browser automatizado en sí en DriverFactory.java, donde especificamos que tipos de preferencias este web browser debe tener, que en nuestra evidencia se refiere a los settings iniciales de Google Chrome.

De ahí, pasamos a la sección de base, y creamos lo que es un BasePage, que será la base lógica de las diferentes páginas del browser que manejaremos. Esta base lógica incluye las secciones de cuánto tiempo Selenium debe esperar para hacer acciones automáticas antes de timeout, los ajustes del locator, que es la herramienta que usa Selenium para manejar la página de browser en sí, y las lógicas de comportamientos de esta misma herramienta. En nuestra evidencia vimos los comportamientos de click, write, text, selectByText, isSelected y exists, viendo como podemos luego expandir esta lista para agregar cualquier lógica que se nos necesite.

De ahí, ya podemos empezar a definir lo que es la sección de páginas en sí, que por estándar lo podemos pensar como instancias de las páginas que vamos a probar, es decir, si queremos probar un sitio en línea que tenga una página dashboard, una página de lista de productos y una página de compras, podemos definirlos como sus propios archivos java para poder probarlos con Selenium. En nuestro caso, nos enfocamos en la sección de Santander Home Page, ya que se nos pidió probar el funcionamiento de acceso del sitio en vivo de Santander a manera de evidencia.

Esto llevo a la creación del SantanderHomePage.java, y adentro lleva las definiciones de la estructura de esta misma. Vemos como definimos el menú de primer nivel de Santander (los tabs de en la parte superior) y incluimos el comportamiento de click de este y sus opciones, junto con una lógica para revisión del url.

Teniendo esto terminado, pasamos a la sección de test, en donde siguiendo el patrón de base, creamos un `BaseTest.java`, dando la base lógica que todas nuestras pruebas necesitan. Definimos que nuestra prueba cree una instancia de Chrome, que métodos deben ocurrir al inicio de cada suite de pruebas con `BeforeClass`, que métodos deben ser antes de cada prueba individual con `BeforeMethod`, y que métodos deben llamarse al terminar el test, que es el cerrado de la instancia de Chrome.

Con esta parte genérica ya definida, ya creamos nuestra base para la creación de nuestras pruebas específica. Llamándolo `SantanderTest.java`, y usando lo que ya aprendimos con anterioridad sobre pruebas con los Asserts y parecidos, definimos en sí lo que queremos comprobar en su propio test. Ya que debíamos revisar si el acceso a las 60 diferentes opciones que tenía Santander que tenía en su primer nivel, cree `DataProviders` (una herramienta incluida en TestNG) y dependiendo de que sección estábamos, ya sea Personas con sus mas de 30 opciones, Empresas, PyMes, Banca Privada, y Acerca de, se puede ir metiendo esas opciones en esa estructura de datos.

Se creó una prueba por sección y creando un assert para cada una revisando que el url al que llegamos sea el correcto. Esta prueba, al agregarle el `dataProvider`, crea una prueba por cada objeto que este provider contenga y evitando crear 60 pruebas casi idénticas, que solamente cambiaban de identificadores.

Al final, al elegir con click derecho el archivo `testng.xml` en la raíz de la carpeta de proyecto, que define que parámetros debe correr las pruebas (Y si falla esto, correr directamente el archivo de pruebas `SantanderTest.java` con `Run As TestNG test`) vemos como nuestro programa empieza a cargar la lógica, vemos como abre un web browser Chrome automatizado, como tiene acceso a la pagina de Santander, prueba su funcionamiento, y al final nos saca en la carpeta `test-output` los resultados en sí además de aparecer en la consola en sí, con un pequeño reporte en `index.html` que nos indica que de nuestras 60 pruebas todas pasaron y demás.